

หัวข้อโครงการ	AI ดันเจี้ยนมาสเตอร์สำหรับการเล่นเทเบิลท็อป RPG แบบแคชวล
หน่วยกิต	6
ผู้เขียน	นายภนลภัส สุทธิมาลา
อาจารย์ที่ปรึกษา	ผศ.ดร. สุริยา นักสุภักพงศ์ ดร. ไพสิฐ ชันอาสา
หลักสูตร	วิศวกรรมศาสตรบัณฑิต
สาขาวิชา	วิศวกรรมหุ่นยนต์และระบบอัตโนมัติ
คณะ	สถาบันวิทยาการหุ่นยนต์ภาคสนาม
ปีการศึกษา	2568

บทคัดย่อ

งานวิจัยนี้มีวัตถุประสงค์เพื่อพัฒนาต้นแบบระบบปัญญาประดิษฐ์ที่ทำหน้าที่เป็น Dungeon Master (AI DM) สำหรับการเล่นเกม Tabletop RPG ในกลุ่มผู้เล่นแคชวล เพื่อลดข้อจำกัดในการที่ต้องพึ่งพา Dungeon Master ที่เป็นมนุษย์ซึ่งต้องอาศัยทั้งความรู้ความเข้าใจในกฎกติกาและการเตรียมตัวล่วงหน้าเป็นอย่างมาก ระบบที่พัฒนาขึ้นจะมุ่งเน้นการสร้างประสบการณ์การเล่นที่เข้าถึงง่ายและเป็นธรรมชาติ โดยจะประกอบด้วยกระบวนการหลักคือ การประมวลผลคำสั่งร่วมกับแบบจำลองภาษาขนาดใหญ่ (LLM) เพื่อสร้างสรรค์เนื้อเรื่องและสถานการณ์ในเกม ระบบดังกล่าวจะถูกกำกับด้วย Rules Engine ที่ออกแบบตามกติกา “Lite 5e” เพื่อรักษาความสมดุลและความถูกต้องของกลไกเกม พร้อมทั้งมีระบบ Game State Tracker สำหรับติดตามสถานะต่างๆ ของผู้เล่นและเกมอย่างต่อเนื่อง เช่น ค่าพลังชีวิต ไอเทม และตำแหน่ง นอกจากนี้ ระบบจะแสดงผลผ่านส่วนต่อประสานกับผู้ใช้ (UI) ที่เรียบง่ายซึ่งออกแบบมาเพื่อการใช้งานบนอุปกรณ์เดียว พร้อมฟังก์ชันเสริม (Optional Functions) ในการรับคำสั่งเสียง (ASR) และการอ่านออกเสียงคำบรรยาย (TTS) เพื่อเป็นทางเลือกในการโต้ตอบ

คำสำคัญ : เอไอ ดันเจี้ยนมาสเตอร์ / เทเบิลท็อปอาร์พีจี / แบบจำลองภาษาขนาดใหญ่ (LLM) / การรู้จำเสียงพูดอัตโนมัติ (ASR) / การแปลงข้อความเป็นเสียง (TTS) / เอ็นจิน์กฎกติกา / ระบบติดตามสถานะเกม / การประมวลผลภาษาธรรมชาติ

Project Title	AI Dungeon Master for Casual Tabletop RPG Play
Project Credits	6
Candidate	Mr. Pollapaat Suttimala
Project Advisor	Asst.Prof.Dr. Suriya Natsupakpong Dr. Paisit Khanarsa
Program	Bachelor of Engineering
Field of Study	Robotics and Automation Engineering
Faculty	Institute of Field Robotics
Academic Year	2025

Abstract

This project aims to develop a prototype Artificial Intelligence system that functions as a Dungeon Master (AI DM) for casual Tabletop RPG players. The primary goal is to mitigate the constraints of relying on a human Dungeon Master, which requires extensive rule knowledge and significant preparation time. The developed system focuses on creating an accessible and natural gameplay experience. The core process involves processing commands in conjunction with a Large Language Model (LLM) to generate narrative content and in-game scenarios. This system is governed by a Rules Engine designed according to "Lite 5e" rules to maintain game balance and mechanical accuracy. Additionally, a Game State Tracker continuously monitors various player and game statuses, such as hit points, items, and location. Finally, the system presents information through a simple User Interface (UI) designed for single-device usability, which also includes optional functions for Automatic Speech Recognition (ASR) and Text-to-Speech (TTS) as alternative interaction methods.

Keywords : AI Dungeon Master / Tabletop RPG / Large Language Model (LLM) / Automatic Speech Recognition (ASR) / Text-to-Speech (TTS) / Rules Engine / Game State Tracker / Natural Language Processing

บทที่ 1 บทนำ

1.1 ความสำคัญและที่มาของงานวิจัย

ในช่วงไม่กี่ปีที่ผ่านมา กระแสความนิยมในเกมกระดาน (Board Game) และเกมสวมบทบาทบนโต๊ะ (Tabletop Role-Playing Game หรือ Tabletop RPG) ได้รับความสนใจเพิ่มขึ้นอย่างกว้างขวาง โดยเฉพาะเกมอย่าง Dungeons & Dragons (D&D) ซึ่งเป็นเกมที่ผู้เล่นกลุ่มหนึ่งจะสวมบทบาทเป็นตัวละครนักผจญภัย และดำเนินเรื่องราวตามจินตนาการร่วมกัน โดยมีผู้เล่นอีกคนหนึ่งทำหน้าที่เป็น "ด้นเจี้ยนมาสเตอร์" (Dungeon Master หรือ DM) คอยสร้างโลก บรรยายสถานการณ์ และควบคุมตัวละครอื่นๆ การรับรู้ที่เพิ่มขึ้นผ่านสื่อออนไลน์และผู้มีอิทธิพลในวงการเกม ได้ดึงดูดให้มีผู้เล่นหน้าใหม่และกลุ่มผู้เล่นทั่วไป (Casual Player) จำนวนมากที่ต้องการสัมผัสประสบการณ์การเล่าเรื่องและการผจญภัยอันเป็นเอกลักษณ์ของเกมประเภทนี้

อย่างไรก็ตาม การเข้าถึงประสบการณ์การเล่น Tabletop RPG ยังคงมีอุปสรรคสำคัญอยู่ที่บทบาทของ ด้นเจี้ยนมาสเตอร์ ซึ่งเปรียบเสมือนหัวใจหลักของเกม การทำหน้าที่ DM ให้สำเร็จลุล่วงได้นั้นต้องอาศัยทักษะและความทุ่มเทหลายด้าน ไม่ว่าจะเป็นความเข้าใจในกฎกติกาที่ซับซ้อน, ความคิดสร้างสรรค์ในการด้นสดและปรับเปลี่ยนเนื้อเรื่องตามการกระทำของผู้เล่น, และที่สำคัญที่สุดคือเวลาในการเตรียมการล่วงหน้าเป็นอย่างมาก ตั้งแต่การสร้างแผนที่ การออกแบบสถานการณ์ ไปจนถึงการเตรียมข้อมูลของศัตรูและตัวละครต่างๆ ซึ่งอาจใช้เวลาเตรียมการหลายวันสำหรับเซสชันการเล่นเพียงครั้งเดียว ข้อจำกัดเหล่านี้ทำให้การเล่นในลักษณะที่ไม่ได้นัดหมายล่วงหน้า หรือการเล่นในบรรยากาศสบายๆ เช่น ที่ร้านกาแฟหรือในงานสังสรรค์ ซึ่งเป็นที่ต้องการของผู้เล่นกลุ่มแคชชวล เป็นไปได้ยากอย่างยิ่ง

แม้ว่าในปัจจุบันจะมีเครื่องมือที่ใช้ปัญญาประดิษฐ์ (AI) เข้ามาเป็นทางเลือก เช่น แอปพลิเคชันช่วยเล่าเรื่องอย่าง AI Dungeon หรือการสนทนากับแชทบอททั่วไป แต่เครื่องมือเหล่านี้ยังเป็นเพียงผู้ช่วยเล่าเรื่องที่เน้นการสร้างบทสนทนาโต้ตอบเท่านั้น และยังไม่สามารถมอบประสบการณ์การเล่น "เกม" ได้อย่างสมบูรณ์ เนื่องจากขาดองค์ประกอบหลักที่สำคัญ ได้แก่ การบังคับใช้กฎกติกาเพื่อรักษาความสมดุลและความท้าทาย, กลไกการทอยลูกเต๋าเพื่อวัดผลการกระทำ, และระบบติดตามสถานะของเกม (Game State) ที่ชัดเจน เช่น ค่าพลังชีวิต ไอเทม และตำแหน่งของตัวละคร ส่งผลให้ผลลัพธ์ที่ได้ใกล้เคียงกับการสนทนาโต้ตอบทั่วไปมากกว่าการเล่นเกม นอกจากนี้ ระบบดังกล่าวยังมีข้อจำกัดด้านการจดจำบริบทและความต่อเนื่องของเนื้อเรื่องในระยะยาว (Long-term Memory) ทำให้ไม่สามารถรักษาสถานะของเกมได้อย่างน่าเชื่อถือ

ดังนั้น โครงการนี้จึงมีเป้าหมายในการพัฒนาต้นแบบระบบ "ปัญญาประดิษฐ์ดันเจี้ยนมาสเตอร์" (AI Dungeon Master) สำหรับผู้เล่นกลุ่มแคชชวลโดยเฉพาะ เพื่อลดช่องว่างและอุปสรรคดังกล่าว โดยเป็นการบูรณาการเทคโนโลยีหลายส่วนเข้าด้วยกันเพื่อเอาชนะข้อจำกัดของเครื่องมือที่มีอยู่เดิม หัวใจหลักของระบบคือการผสานการทำงานระหว่างแบบจำลองภาษาขนาดใหญ่ (LLM) ซึ่งทำหน้าที่สร้างสรรค์เนื้อเรื่อง กับ "เอนจินกฎกติกา" (Rules Engine) ที่สร้างขึ้นตามกฎ "Lite 5e" (ดูภาคผนวก ก) เพื่อควบคุมความถูกต้องและความสมดุลของกลไกเกม ระบบจะมี "ระบบติดตามสถานะเกม" (Game State Tracker) เพื่อจัดการข้อมูลสำคัญอย่างต่อเนื่อง และนำเสนอผ่านส่วนต่อประสานกับผู้ใช้ (UI) ที่เรียบง่าย นอกจากนี้ ยังมีการผนวกเทคโนโลยีสนับสนุน เช่น ระบบการรู้จำเสียงพูด (ASR) เพื่อเป็นทางเลือกในการโต้ตอบ เป้าหมายสูงสุดของโครงการคือการลดการพึ่งพา Dungeon Master ที่เป็นมนุษย์ แต่ยังคงรักษาแก่นแท้ของความสมดุล ความท้าทาย และความสนุกของการเล่น Tabletop RPG ไว้ได้อย่างครบถ้วน

1.2 วัตถุประสงค์ของงานวิจัย

1. เพื่อพัฒนาต้นแบบระบบ AI Dungeon Master ที่ใช้แบบจำลองภาษาขนาดใหญ่ (LLM) ในการดำเนินเรื่องราวและเหตุการณ์ ภายใต้การกำกับของ Rules Engine ที่แม่นยำตามกติกาแบบ Lite 5e
2. เพื่อออกแบบและพัฒนา Rules Engine และ Game State Tracker ที่สามารถบังคับใช้และติดตามกลไกหลักของเกมได้อย่างถูกต้องและต่อเนื่อง เช่น การตรวจสอบค่าความสามารถ, การโจมตี, ค่าความเสียหาย, พลังชีวิต และสถานะต่างๆ ของตัวละคร
3. เพื่อพัฒนาส่วนต่อประสานกับผู้ใช้ (UI) ที่ใช้งานง่ายและตอบสนองต่อการเล่นจริง ประกอบด้วยหน้าต่างแสดงข้อมูลตัวละคร (Character Sheet), แผนที่ (Map), และบันทึกเหตุการณ์ (Log) เพื่อรองรับการเล่นบนอุปกรณ์เดียว รวมถึงบูรณาการเทคโนโลยีสนับสนุน (ASR และ TTS) เพื่อเป็นทางเลือกในการโต้ตอบกับระบบ

1.3 ประโยชน์และผลที่คาดว่าจะได้รับจากงานวิจัย

1. ได้รับต้นแบบระบบ AI Dungeon Master ที่สามารถใช้งานได้จริงสำหรับผู้เล่นกลุ่มแคชชวล ช่วยลดอุปสรรคในการเริ่มต้นเล่น Tabletop RPG
2. ได้รับแนวทางการบูรณาการเทคโนโลยีแบบจำลองภาษาขนาดใหญ่ (LLM) เข้ากับระบบเชิงตรรกะ (Rules Engine, State Tracker) สำหรับประยุกต์ใช้กับเกมที่มีโครงสร้างและกฎเกณฑ์ชัดเจน รวมถึงแนวทางการผนวกเทคโนโลยีสนับสนุน (ASR, TTS) เข้ากับระบบ

3. ได้รับระบบที่ช่วยยกระดับประสบการณ์การเล่นในบริบทที่ไม่เป็นทางการให้สะดวกและเข้าถึงง่ายยิ่งขึ้น
4. ได้รับองค์ความรู้เชิงวิศวกรรมที่เป็นรากฐานสำคัญในการต่อยอดโครงการในอนาคต เช่น การพัฒนาให้รองรับการเล่นหลายคน (Multiplayer) ผ่านเครือข่าย หรือการพัฒนาให้รองรับภาษาไทย

1.4 ขอบเขตงานวิจัย

1.4.1 ด้านระบบเกมและกติกา

1. ระบบรองรับการเล่น Tabletop RPG โดยใช้ชุดกติกาแบบ "Lite 5e" ซึ่งเป็นกฎที่คณะผู้จัดทำได้นิยามและกำหนดขอบเขตขึ้นโดยเฉพาะ (ดูรายละเอียดฉบับเต็มในภาคผนวก ก) โดยได้ปรับลดความซับซ้อนลง แต่ยังคงรักษากฎหลักที่จำเป็นไว้ เช่น การตรวจสอบค่าความสามารถ (d20 check), การโจมตีและความเสียหาย, ค่าพลังชีวิต (HP), และการเคลื่อนที่แบบโซน (Zone-based Movement) ขอบเขตของกติกาเกมที่ระบบรองรับจะจำกัดอยู่ภายใต้กฎ Lite 5e นี้เท่านั้น

2. ระบบถูกออกแบบมาเพื่อรองรับการเล่นเนื้อเรื่องสั้นที่จบในตัว (One-shot Session) และมีระบบบันทึกสถานะเกม (Save/Load) เพียงหนึ่งช่องสำหรับเล่นต่อเนื่อง

3. รูปแบบการเล่นและการตัดสินผล ระบบนี้ใช้สถาปัตยกรรมแบบ 'Hybrid-Arbitrated' ซึ่งแบ่งการประมวลผลคำสั่งของผู้เล่นเป็น 2 ประเภทหลัก เพื่อให้มีความยืดหยุ่นเหมือนเล่นกับมนุษย์ แต่ยังคงความแม่นยำของกฎไว้

1) การกระทำแบบมีกฎตายตัว (Fixed Actions): คือการกระทำที่มีกฎชัดเจนใน Lite 5e Rulebook (ภาคผนวก ก) เช่น 'การโจมตี' (Attacking), 'การเคลื่อนที่' (Movement), หรือ 'การใช้ความสามารถ' (Use Ability) คำสั่งเหล่านี้จะถูก Rules Engine (ส่วนที่เป็นโค้ด) ประมวลผลก่อนเพื่อคำนวณผลลัพธ์ (เช่น Hit/Miss, Damage) จากนั้น LLM จะทำหน้าที่บรรยายผลลัพธ์ ที่คำนวณได้

2) การกระทำเชิงสร้างสรรค์ (Creative Actions): คือการกระทำที่อยู่นอกเหนือกฎตายตัวที่ระบุไว้ชัดเจน เช่น 'การสำรวจสภาพแวดล้อม' (Exploration), 'การไขปริศนา' (Puzzles), หรือการกระทำที่ผู้เล่นคิดขึ้นเอง (เช่น "เผาหีบเพื่อละลายกัญญาเจ") คำสั่งเหล่านี้จะถูกส่งให้ LLM (ในฐานะ DM) ตีความและทำการตัดสินใจก่อน (Arbitration) เช่น กำหนดค่าความยาก (DC) หรือระบุค่าสถานะ (PHYS, MENT, SOC) ที่ต้องใช้ในการทอยลูกเต๋า จากนั้น Rules Engine จึงจะทำการทอยลูกเต๋าตามที่ LLM กำหนด และ LLM จะบรรยายผลลัพธ์ ตามผลการทอยนั้น

4. ขอบเขตการทำงานของต้นแบบ (Prototype Scope): เพื่อให้สามารถพัฒนาได้ทันตามกำหนดเวลา และตอบสนองต่อข้อเสนอแนะในการลดขอบเขต โครงการนี้จะมุ่งเน้นการพัฒนาตามขอบเขตดังตารางต่อไปนี้

In-Scope (สิ่งที่ทำในโครงการนี้)	Out-of-Scope (สิ่งที่ยังไม่ทำในโครงการนี้)
1. Fixed Actions: กลไกการต่อสู้ (Combat) และการกระทำหลักตามที่ระบุใน Lite 5e Rulebook (ภาคผนวก ก) อย่างครบถ้วน	1. Complex Rules: กฎกติกาที่ซับซ้อนนอกเหนือจาก Lite 5e (เช่น กฎการลอบเร้นขั้นสูง, การโจมตีฉวยโอกาส, สถานะผิดปกติอื่นๆ นอกเหนือจากที่ระบุ)
2. Creative Actions: การสำรวจ (Exploration) และการไขปริศนา (Puzzles) ที่สามารถตัดสินใจได้ด้วยการทอย 3 ค่าสถานะหลัก (PHYS, MENT, SOC) ตามที่ LLM กำหนด DC	2. Complex Scenarios: ปริศนาแบบหลายขั้นตอน (Multi-stage puzzles), บทสนทนาที่แตกแขนง (Branching dialogues) ที่ซับซ้อน, หรือการสืบสวนสอบสวน
3. State Tracking: ติดตามเฉพาะสถานะที่จำเป็นต่อการเล่นตามกฎ Lite 5e (HP, Zone, Items พื้นฐาน, สถานะผิดปกติที่ระบุไว้)	3. Unforeseen Events: เหตุการณ์สุ่ม (Random Events), Quick-Time Events, หรือกลไกที่เกิดขึ้นนอกกรอบการเล่นของผู้เล่น
4. Scenario Content: รวบรวมการเล่นใน 1 สถานการณ์ทดสอบหลัก (Pilot Test Scenario) ที่ออกแบบไว้ล่วงหน้า (ลักษณะ Mission-based)	4. Dynamic World: โลกที่เปลี่ยนแปลงตลอดเวลา, NPC ที่มีความสัมพันธ์ซับซ้อน, หรือระบบเศรษฐกิจภายในเกม

ตารางที่ X ขอบเขตการทำงานของต้นแบบ

1.4.2 ด้านการปฏิสัมพันธ์กับผู้ใช้

- ระบบรองรับอินพุตหลักผ่านการพิมพ์ (Text Input) และมีอินพุตทางเลือก (Optional Input) เป็นคำสั่งเสียงภาษาอังกฤษ ซึ่งจะถูกละเปลี่ยนเป็นข้อความผ่านโมดูล ASR
- ส่วนต่อประสานกับผู้ใช้ (UI) ถูกออกแบบมาเพื่อใช้งานบนอุปกรณ์เดียว และมีองค์ประกอบหลักดังนี้
 - การตั้งค่าเริ่มต้น ผู้เล่นสามารถกำหนดจำนวนตัวละคร (1-4 ตัว) และเลือกธีมของโลก/เนื้อเรื่อง (เช่น แฟนตาซี, ไซไฟ) ก่อนเริ่มเกม

2) การควบคุมตัวละคร มีกลไกให้ผู้ใช้เลือกตัวละครที่ต้องการจะสั่งการก่อนป้อนคำสั่ง (พูดหรือพิมพ์) เพื่อให้ระบบสามารถระบุผู้กระทำได้อย่างถูกต้อง

3) หน้าต่างแสดงข้อมูล (Character Sheet) แสดงค่าสถานะ, พลังชีวิต และความสามารถหลักของตัวละคร

4) แผนที่แบบโซน (Zone Map) แสดงตำแหน่งของผู้เล่นและศัตรูในลักษณะโซน (เช่น ระเบียงใกล้, กลาง, ไกล) แทนการใช้ตาราง

5) บันทึกเหตุการณ์ (Action Log) แสดงผลการกระทำ, ผลการทอยลูกเต๋า และการตัดสินใจตามกติกาของ Rules Engine อย่างโปร่งใส

1.4.3 ด้านเทคนิคและการวัดผล

1. ระบบจะใช้บริการ LLM ผ่าน API เป็นหลัก โดยไม่เน้นการพัฒนากราฟิกสามมิติหรือแอนิเมชันที่ซับซ้อน แต่ให้ความสำคัญกับความเสถียรและความเข้าใจง่ายในการเล่น
2. ความสำเร็จของต้นแบบจะถูกวัดผลผ่านเกณฑ์เชิงหน้าที่ (Functional Criteria) ดังนี้
 - 1) ความถูกต้องของกฎ (Rule Adherence) Rules Engine ต้องสามารถตัดสินใจและคำนวณผลลัพธ์ตามกฎ Lite 5e ที่กำหนดไว้ได้อย่างถูกต้องแม่นยำ
 - 2) ความสมบูรณ์ของสถานะ (State Integrity) Game State Tracker ต้องสามารถบันทึกและปรับปรุงข้อมูลสถานะของเกมได้อย่างถูกต้อง ไม่เกิดข้อผิดพลาดร้ายแรงตลอดการเล่น
 - 3) การจัดการลำดับการเล่น (Flow Management): ระบบต้องสามารถนำเสนอทางเลือกและการกระทำที่ชัดเจน ทำให้ผู้เล่นสามารถดำเนินเกมไปในแต่ละรอบ (Turn) จนจบได้อย่างราบรื่นโดยไม่เกิดสถานะหยุดชะงักที่เกิดจากตัวระบบเอง

1.5 ข้อตกลงเบื้องต้น

1. ในการดำเนินโครงการนี้จะใช้ โมเดลโอเพนซอร์สสำเร็จรูป สำหรับระบบการรู้จำเสียงพูด (ASR) และใช้บริการ API (Application Programming Interface) จากผู้ให้บริการภายนอกสำหรับแบบจำลองภาษาขนาดใหญ่ (LLM) โดยจะเลือกใช้โมเดลที่มีค่าใช้จ่ายไม่สูง (เช่น Gemini Flash)
2. ผู้ใช้งานระบบจำเป็นต้องมี API Key ของตนเองสำหรับ LLM และรับผิดชอบค่าใช้จ่ายที่เกิดขึ้นจากการใช้บริการ API ดังกล่าว โดยใน UI จะมีช่องสำหรับผู้กรอก API Key
3. ต้นแบบของระบบที่พัฒนาขึ้นจะมุ่งเน้นประสบการณ์การเล่นบนอุปกรณ์เดียว (Single-device experience) โดยผู้เล่นหนึ่งคนสามารถควบคุมตัวละครได้หลายตัว หรือผู้เล่นหลายคนสามารถสลับกันเล่นได้

- ชุดกติกา "Lite 5e" ที่ใช้ในโครงการจะเป็นชุดกติกาที่คณะผู้จัดทำได้นิยามและกำหนดขอบเขตขึ้น โดยอ้างอิงจากกลไกหลักของเกม Dungeons & Dragons ฉบับที่ 5

1.6 ขั้นตอนการดำเนินงาน

ในกระบวนการดำเนินงานวิจัยนี้ ได้ทำการแบ่งขั้นตอนการดำเนินงานเพื่อเพิ่มประสิทธิภาพของงาน โดยมีรายละเอียดดังนี้

1. ทบทวนทฤษฎีและงานวิจัยที่เกี่ยวข้องกับเทคโนโลยีที่จะนำมาใช้ในโครงการ
2. กำหนดคุณสมบัติของระบบที่ต้องการ (Requirements) และเกณฑ์ชี้วัดความสำเร็จ (Success Criteria)
3. ออกแบบสถาปัตยกรรมของระบบ (System Architecture) และแผนผังการไหลของข้อมูล (Data Flow)
4. พัฒนาต้นแบบขั้นที่หนึ่ง (Prototype-A) ซึ่งประกอบด้วยการเชื่อมต่อระบบ LLM, Rules Engine เบื้องต้น, และระบบบันทึกเหตุการณ์ (Log)
5. พัฒนา Rules Engine และ Game State Tracker
6. พัฒนาส่วนต่อประสานกับผู้ใช้ (UI) ซึ่งประกอบด้วย Character Sheet, Zone Map, และ Battle Log
7. บูรณาการทุกส่วนประกอบเป็นต้นแบบขั้นที่สอง (Prototype-B), ผนวกฟังก์ชันเสริม (ASR/TTS), และทำการทดสอบการทำงานเบื้องต้น
8. ทดลองใช้งานระบบ (Pilot Test) เพื่อรวบรวมผลตอบรับและข้อเสนอแนะสำหรับนำมาปรับปรุง
9. ปรับปรุงแก้ไขระบบขั้นสุดท้าย และจัดทำรายงานโครงการฉบับสมบูรณ์

1.7 นิยามศัพท์เฉพาะ

1. AI Dungeon Master (AI DM) หมายถึง ระบบปัญญาประดิษฐ์ที่ทำหน้าที่เป็นผู้ดำเนินเกมและผู้ตัดสินในเกม Tabletop RPG
2. Tabletop RPG (เทเบิลท็อปอาร์พีจี) หมายถึง เกมสวมบทบาทที่เล่นบนโต๊ะ โดยอาศัยการเล่าเรื่อง การทอยลูกเต๋า และจินตนาการของผู้เล่นเป็นหลัก
3. Large Language Model (LLM) หมายถึง แบบจำลองภาษาขนาดใหญ่ เป็นปัญญาประดิษฐ์ที่เชี่ยวชาญด้านการเข้าใจและสร้างข้อความที่เหมือนมนุษย์
4. Rules Engine (เอ็นจินกฎกติกา) หมายถึง ส่วนประกอบของซอฟต์แวร์ที่สร้างขึ้นเพื่อบังคับใช้กฎกติกาและกลไกของเกมโดยเฉพาะ

5. Game State Tracker (ระบบติดตามสถานะเกม) หมายถึง ส่วนประกอบของซอฟต์แวร์ที่ทำหน้าที่บันทึกและจัดการข้อมูลทั้งหมดที่เกิดขึ้นในเกมแบบเรียลไทม์
6. Lite 5e หมายถึง ชุดกติกาที่ถูกปรับลดความซับซ้อนลงจาก Dungeons & Dragons ฉบับที่ 5 เพื่อให้เหมาะกับผู้เล่นกลุ่มแคชวล
7. Automatic Speech Recognition (ASR) หมายถึง เทคโนโลยีการรู้จำเสียงพูดอัตโนมัติ ทำหน้าที่แปลงสัญญาณเสียงพูดให้เป็นข้อความ
8. Text-to-Speech (TTS) หมายถึง เทคโนโลยีการแปลงข้อความที่เป็นเสียงสังเคราะห์ ทำหน้าที่อ่านออกเสียงคำบรรยายของระบบเพื่อเป็นทางเลือกในการรับฟัง

บทที่ 2 ทฤษฎี/งานวิจัยที่เกี่ยวข้อง

ในบทนี้จะกล่าวถึงทฤษฎีพื้นฐานและเทคโนโลยีที่เกี่ยวข้องซึ่งเป็นหัวใจสำคัญในการพัฒนาโครงการ "AI Dungeon Master" รวมถึงทบทวนงานวิจัยและโครงการที่มีอยู่ก่อนหน้านี้ เพื่อวิเคราะห์แนวทางการพัฒนา จุดแข็ง จุดอ่อน และนำมาเป็นแนวทางในการออกแบบสถาปัตยกรรมของระบบในโครงการนี้ เนื้อหาจะแบ่งออกเป็นสองส่วนหลัก ได้แก่ ส่วนของทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง และส่วนของงานวิจัยที่เกี่ยวข้อง

2.1 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

โครงการนี้เป็นการบูรณาการเทคโนโลยีปัญญาประดิษฐ์หลายแขนงเข้าด้วยกัน เพื่อสร้างระบบที่สามารถทำหน้าที่อันซับซ้อนของ Dungeon Master ได้ เทคโนโลยีหลักที่นำมาใช้ประกอบด้วย แบบจำลองภาษาขนาดใหญ่ (LLM) และสถาปัตยกรรมปัญญาประดิษฐ์แบบผสมผสาน (Hybrid AI Architecture) โดยมีเทคโนโลยีสนับสนุน (Supporting Technologies) ได้แก่ การรู้จำเสียงพูดอัตโนมัติ (ASR) และการแปลงข้อความเป็นเสียง (TTS)

2.1.1 แบบจำลองภาษาขนาดใหญ่ (Large Language Models - LLM)

แบบจำลองภาษาขนาดใหญ่ หรือ LLM คือแบบจำลองปัญญาประดิษฐ์ประเภทหนึ่งที่ได้รับการฝึกฝนด้วยข้อมูลข้อความจำนวนมาก ทำให้มีความสามารถสูงในการทำความเข้าใจและสร้างภาษาที่ใกล้เคียงกับมนุษย์ แบบจำลองที่มีชื่อเสียงในปัจจุบัน เช่น GPT (Generative Pre-trained Transformer) ของ OpenAI หรือ Gemini ของ Google ล้วนมีพื้นฐานมาจากสถาปัตยกรรมที่เรียกว่า Transformer ซึ่งเป็นปัจจัยสำคัญที่ทำให้ LLM สามารถจัดการกับความซับซ้อนของภาษาได้ดี

1. จุดแข็งสำหรับโครงการ

1) การสร้างสรรค์เนื้อหา (Creative Text Generation) จุดแข็งที่สำคัญที่สุดของ LLM คือความสามารถในการสร้างเรื่องราว, คำบรรยายฉาก, และบทสนทนาของตัวละคร (NPC) ได้อย่างสร้างสรรค์และต่อเนื่อง ทำให้โลกของเกมมีความสมจริงและน่าสนใจ การทำงานแบบ Probabilistic ซึ่งอิงตามรูปแบบภาษาที่ได้เรียนรู้มา ช่วยให้ LLM สามารถสร้างผลลัพธ์ที่หลากหลายและคาดไม่ถึง ซึ่งเป็นหัวใจของความสนุกในเกม Tabletop RPG ที่ผู้เล่นมักจะกระทำการนอกกรอบที่คาดไว้ ตัวอย่างเช่น หากผู้เล่นตัดสินใจปีนหน้าต่างของโรงเตี๊ยมแทนที่จะเดินเข้าประตูหน้า LLM สามารถ

สร้างคำบรรยายถึงสภาพของหน้าต่างที่เก่าแก่ เสียงไม้ที่ลั่นเอี๊ยด และลมเย็นที่พัดเข้ามาได้ในทันทีโดยไม่ต้องมีการเตรียมสคริปต์ไว้ล่วงหน้า

2) การจัดการบทสนทนา (Dialogue Management) LLM สามารถเข้าใจเจตนาของผู้เล่นจากคำสั่งที่เป็นภาษาธรรมชาติ และสร้างการตอบสนองที่สอดคล้องกับสถานการณ์ได้อย่างยืดหยุ่น ซึ่งแตกต่างอย่างสิ้นเชิงจากระบบบทสนทนาแบบต้นไม้ (Dialogue Tree) ในเกมดั้งเดิมที่ผู้เล่นถูกจำกัดให้เลือกตอบตามตัวเลือกที่กำหนดไว้ล่วงหน้าเท่านั้น ความสามารถนี้ทำให้ตัวละครในเกม (NPC) มีชีวิตชีวาและตอบสนองได้อย่างสมจริง ตัวอย่างเช่น แทนที่จะมีเพียงตัวเลือกสนทนา 3 ข้อ ผู้เล่นสามารถถามคำถามปลายเปิดกับ NPC เช่น "คุณเคยได้ยินข่าวลือเกี่ยวกับมังกรบนภูเขาบ้างไหม?" และ LLM ก็สามารถสร้างคำตอบที่สอดคล้องกับบทบาทและบุคลิกของ NPC ตัวนั้นได้ทันที

2. ข้อจำกัดและประเด็นที่ต้องพิจารณา

1) ภาวะไร้สถานะ (Statelessness) และข้อจำกัดของหน่วยความจำ: LLM ไม่มีหน่วยความจำถาวร มันจะจดจำข้อมูลได้เท่าที่มีอยู่ใน "หน้าต่างบริบท" (Context Window) เท่านั้น ซึ่งเป็นเหมือนหน่วยความจำระยะสั้นที่มีขนาดจำกัด เมื่อการสนทนาหรือเรื่องราวดำเนินไปนานขึ้น ข้อมูลในช่วงแรกๆ จะหลุดออกจากหน้าต่างบริบท ทำให้ LLM อาจ "ลืม" เหตุการณ์สำคัญ ชื่อตัวละคร หรือข้อมูลที่เคยเกิดขึ้นไปแล้วได้ ปัญหานี้เป็นอุปสรรคสำคัญต่อการสร้างเรื่องราวที่ต่อเนื่องและสมเหตุสมผลในระยะยาว

2) การยึดถือกฎกติกา (Rule Adherence): โดยธรรมชาติแล้ว LLM เป็นแบบจำลองเชิงความน่าจะเป็น (Probabilistic Model) ไม่ใช่ระบบเชิงตรรกะ (Logical System) ทำให้มันไม่สามารถรับประกันได้ว่าจะปฏิบัติตามกฎกติกาที่ซับซ้อนและตายตัวของเกมได้อย่างถูกต้อง 100% บ่อยครั้งที่ LLM อาจสร้างผลลัพธ์ที่ดูสมเหตุสมผลแต่ผิดหลักกลไกของเกม หรือที่เรียกว่า "Hallucination" ซึ่งเป็นสิ่งที่ไม่สามารถยอมรับได้ในบริบทของเกมที่ต้องการความยุติธรรม

ข้อจำกัดทั้งสองข้อนี้เป็นความท้าทายหลักเชิงสถาปัตยกรรม และเป็นเหตุผลสำคัญที่โครงการนี้จำเป็นต้องมีส่วนประกอบอื่นเข้ามาทำงานร่วมกับ LLM แทนที่จะใช้ LLM เพียงอย่างเดียว

2.1.2 การรู้จำเสียงพูดอัตโนมัติ (Automatic Speech Recognition - ASR)

ASR คือเทคโนโลยีที่ทำหน้าที่แปลงสัญญาณเสียงพูดของมนุษย์ให้กลายเป็นข้อมูลข้อความดิจิทัล เพื่อให้คอมพิวเตอร์สามารถนำไปประมวลผลต่อได้ ในบริบทของโครงการนี้ ASR จะถูกนำมาใช้เป็น "ฟังก์ชันเสริม" (Optional Function) เพื่อเป็น "ทางเลือก" ให้ผู้เล่นสามารถสั่งการตัวละครด้วยเสียงพูดได้ นอกเหนือจากการพิมพ์

ในปัจจุบัน เทคโนโลยี ASR สามารถแบ่งได้เป็น 2 ประเภทหลัก คือแบบที่อาศัยบริการคลาวด์ (Cloud-based API) และแบบโมเดลโอเพนซอร์สที่ประมวลผลบนอุปกรณ์ของผู้ใช้โดยตรง (On-device) ในปัจจุบันมีโมเดลโอเพนซอร์สประสิทธิภาพสูงที่ประมวลผลบนอุปกรณ์ (On-device) เช่น Whisper ของ OpenAI หรือ Vosk ซึ่งช่วยรักษาความเป็นส่วนตัวของข้อมูลเสียง ในโครงการนี้จะทบทวนแนวทางดังกล่าวเพื่อนำมาบูรณาการ (Integrate) เป็นฟังก์ชันเสริม โดยมุ่งเน้นที่การเชื่อมต่อเป็นทางเลือกให้ผู้ใช้ มากกว่าการวัดผลประสิทธิภาพของตัวโมเดล

2.1.3 การแปลงข้อความเป็นเสียง (Text-to-Speech - TTS)

เช่นเดียวกับ ASR, เทคโนโลยี TTS (Text-to-Speech) จะถูกนำมาใช้เป็นฟังก์ชันเสริม เพื่อเพิ่มอรรถรส (Immersion) ในการเล่น โดยระบบจะอ่านออกเสียงคำบรรยายของ AI DM เป็นทางเลือกให้ผู้เล่น โครงการนี้จะทบทวนไลบรารีมาตรฐานที่มีใน Flutter หรือโมเดลโอเพนซอร์สขนาดเล็ก ที่ใช้ทรัพยากรน้อย โดยเน้นการบูรณาการเป็นฟังก์ชันทางเลือกเท่านั้น ไม่ได้มุ่งเน้นการวัดผลประสิทธิภาพของเสียงสังเคราะห์

2.1.4 สถาปัตยกรรมปัญญาประดิษฐ์แบบผสมผสาน (Hybrid AI Architecture)

เพื่อแก้ไขข้อจำกัดของ LLM ที่กล่าวไปข้างต้น โครงการนี้จึงได้นำแนวคิดของ สถาปัตยกรรมปัญญาประดิษฐ์แบบผสมผสาน หรือที่รู้จักในอีกชื่อหนึ่งว่า Neuro-Symbolic AI มาใช้ แนวคิดนี้คือการนำระบบสองประเภทที่มีความถนัดต่างกันมาทำงานร่วมกัน เปรียบได้กับการทำงานของสมองมนุษย์สองส่วน (System 1 และ System 2 Thinking)

1. Neural Component (LLM) เปรียบเสมือน System 1 คือส่วนของโครงข่ายประสาทเทียมที่ทำงานอย่างรวดเร็ว เป็นธรรมชาติ และใช้สัญชาตญาณ ในที่นี้คือ LLM ซึ่งทำหน้าที่เกี่ยวกับความคิดสร้างสรรค์ การเข้าใจภาษา และการสร้างเนื้อหาเชิงพรรณนา
2. Symbolic Component (Code) เปรียบเสมือน System 2 คือส่วนที่ทำงานอย่างช้าๆ เป็นเหตุเป็นผล และอยู่บนพื้นฐานของตรรกะและกฎเกณฑ์ที่กำหนดไว้อย่างชัดเจน ในโครงการนี้หมายถึง Rules Engine และ Game State Tracker ที่เขียนขึ้นด้วยโค้ดโปรแกรม

สถาปัตยกรรมแบบผสมผสานนี้จะแบ่งหน้าที่ความรับผิดชอบอย่างชัดเจน

1. LLM รับผิดชอบในส่วนของการเล่าเรื่อง (Narrative) เช่น การบรรยายฉาก, การตอบโต้ของ NPC

2. Rules Engine & Game State Tracker รับผิดชอบในส่วนของ "กลไกเกมและหน่วยความจำ" (Mechanics & Memory) เช่น การคำนวณความเสียหาย, การตรวจสอบผลการทอยลูกเต๋า, และการบันทึกว่า "ตอนนี้ผู้เล่นมีพลังชีวิตเหลือเท่าไร"

ด้วยสถาปัตยกรรมนี้ ระบบจะให้ LLM ทำในสิ่งที่ถนัด และใช้โค้ดโปรแกรมที่ทำงานอย่างแม่นยำมาจัดการในส่วนที่ LLM ทำได้ไม่ดี ซึ่งเป็นแนวทางที่ได้รับการยอมรับอย่างกว้างขวางในการพัฒนาระบบ AI ที่ต้องการทั้งความคิดสร้างสรรค์และความน่าเชื่อถือ

2.1.5 การสร้างข้อความด้วยการดึงข้อมูลมาเสริม (Retrieval-Augmented Generation - RAG)

RAG เป็นเทคนิคที่กำลังได้รับความนิยมอย่างสูงในการเพิ่มประสิทธิภาพและความน่าเชื่อถือให้กับ LLM หลักการของ RAG คือ แทนที่จะปล่อยให้ LLM ตอบคำถามจาก "ความรู้" ที่มีอยู่ภายในเพียงอย่างเดียว ระบบจะทำการ ดึงข้อมูลที่เกี่ยวข้อง จากแหล่งข้อมูลภายนอก (เช่น เอกสาร, ฐานข้อมูล) แล้วนำข้อมูลนั้นมาประกอบในคำสั่ง (Prompt) เพื่อให้ LLM ใช้เป็นบริบทในการสร้างคำตอบ ในโครงการนี้ RAG จะเป็นกลไกสำคัญที่เชื่อมต่อระหว่าง LLM และ Symbolic Component

1. การดึงกฎกติกา (Rule Retrieval): เมื่อผู้เล่นต้องการกระทำการบางอย่างที่เกี่ยวข้องกับกฎ เช่น "ร้ายเวทมนตร์ Fireball" ระบบ RAG สามารถดึงข้อมูลกฎของเวทมนตร์ Fireball จากฐานข้อมูล แล้วส่งไปพร้อมกับคำสั่งให้ LLM เพื่อให้มันบรรยายผลลัพธ์ได้ถูกต้องตามกฎ เช่น Damage: 8d6 fire damage, Range: 150 feet
2. การดึงสถานะของเกม (State Retrieval): ในทุกๆ การกระทำของผู้เล่น ระบบ RAG จะดึงข้อมูลสถานะล่าสุดของเกมจาก Game State Tracker (เช่น Player HP: 12/15, Goblin HP: 5/7) มาประกอบในคำสั่ง เพื่อให้ LLM "รับรู้" สถานการณ์ปัจจุบันและสร้างเนื้อเรื่องที่สอดคล้องกับสิ่งที่เกิดขึ้นจริงในเกม เช่น การบรรยายว่าก๊อบลิน "ดูบาดเจ็บสาหัส" เมื่อพลังชีวิตเหลือน้อย

การใช้เทคนิค RAG ทำให้เราสามารถแก้ปัญหา Long-term Memory ของ LLM ได้อย่างมีประสิทธิภาพ และยังเป็นกลไกสำคัญในการกำกับให้ LLM สร้างเนื้อหาที่สอดคล้องกับกฎและสถานะของเกมอยู่เสมอ

2.2 งานวิจัยและโครงการที่เกี่ยวข้อง

ในช่วงไม่กี่ปีที่ผ่านมา มีความพยายามในการนำ LLM มาประยุกต์ใช้เป็น Dungeon Master หลายโครงการ การทบทวนงานวิจัยเหล่านี้ช่วยให้เห็นถึงแนวทาง ความท้าทาย และบทเรียนที่สามารถนำมาปรับใช้กับโครงการนี้ได้

2.2.1 Triyason (2023) - การใช้ ChatGPT เป็น DM สำหรับผู้เล่นคนเดียว

งานวิจัยชิ้นแรกๆ ที่ทดลองใช้ ChatGPT ในการดำเนินเกม D&D พบว่า LLM สามารถสร้างเรื่องราวที่ต่อเนื่องและตอบสนองต่อการกระทำของผู้เล่นได้ดีพอที่จะทำให้ผู้เล่นใหม่ได้สัมผัสกับประสบการณ์ D&D อย่างไรก็ตาม ข้อจำกัดที่พบคือการตอบสนองที่ล่าช้าในบางครั้ง และการขาดการแสดงอารมณ์ร่วม (Limited Emotional Engagement) ซึ่งส่งผลต่อความอินของผู้เล่น งานวิจัยนี้ชี้ให้เห็นถึงศักยภาพของ LLM แต่ก็ยังชี้ถึงความจำเป็นในการปรับปรุงด้านความเร็วและความสามารถในการเล่าเรื่องให้น่าดึงดูดยิ่งขึ้น

2.2.2 Sakellaridis (2024) - การเปรียบเทียบ LLM ที่ผ่านการ Fine-tune กับ DM มนุษย์

งานวิจัยนี้ได้ทำการฝึกฝน (Fine-tune) LLM ด้วยข้อมูลบทสนทนาจากเกม D&D และข้อมูลกฎกติกาโดยเฉพาะ แล้วนำมาเปรียบเทียบกับการเล่นกับ DM ที่เป็นมนุษย์ ผลลัพธ์ที่น่าสนใจคือ ผู้เล่นให้คะแนนความดื่มด่ำกับคำบรรยายจากของ AI DM สูงเทียบเท่าหรือสูงกว่า DM มนุษย์เสียอีก แสดงให้เห็นว่า LLM มีความเป็นเลิศในการสร้างสรรค์คำบรรยาย แต่ข้อเสียที่พบคือ AI มักจะดำเนินเรื่องไปในทิศทางที่กำหนดไว้ล่วงหน้า (Railroading) ไม่ยืดหยุ่นต่อการตัดสินใจนอกกรอบของผู้เล่น และสร้างความท้าทายในเกมได้ไม่ดีพอ

2.2.3 Jørgensen et al. (2024) - สถาปัตยกรรมแบบ Multi-Agent

อีกหนึ่งแนวทางที่น่าสนใจคือการออกแบบ AI DM ด้วยสถาปัตยกรรมแบบหลายเอเจนต์ (Multi-Agent) โดยใช้เฟรมเวิร์กอย่าง ReAct เพื่อให้ AI สามารถ "คิด" และ "กระทำ" เป็นขั้นเป็นตอนได้ โดยอาจมีเอเจนต์ที่ทำหน้าที่เฉพาะทาง เช่น เอเจนต์สำหรับเล่าเรื่อง, เอเจนต์สำหรับจัดการกฎกติกา ซึ่งช่วยให้ระบบโดยรวมทำงานได้อย่างสอดคล้องและน่าเชื่อถือมากขึ้น แนวทางนี้สนับสนุนการออกแบบเชิงโมดูลของโครงการเรา

2.2.4 Tool-Assisted AI DM (2024) - การใช้ RAG และ Function Calling

งานวิจัยล่าสุดได้นำเสนอแนวคิด AI DM ที่สามารถเรียกใช้เครื่องมือภายนอก (External Tools) ได้ เช่น การเรียกฟังก์ชันสำหรับทอยลูกเต๋า (Function Calling) หรือการใช้ RAG เพื่อดึงกฎกติกาจากฐานข้อมูลตามเวลาจริง ผลการทดลองพบว่าสถาปัตยกรรมแบบผสมผสานนี้ช่วยให้ AI DM มีความ

สม่ำเสมอในการรักษาสถานะของเกม และปฏิบัติตามกฎได้อย่างน่าเชื่อถือสูงกว่าการใช้ LLM เพียงอย่างเดียว

2.2.5 สรุปและบทเรียนที่ได้รับจากงานวิจัยที่เกี่ยวข้อง

จากงานวิจัยทั้งหมดที่กล่าวมา สามารถสรุปบทเรียนสำคัญและแนวทางสำหรับโครงการนี้ได้ดังนี้

1. LLM-only ไม่เพียงพอ: งานวิจัยของ Triyason (2023) และ Sakellaridis (2024) แสดงให้เห็นอย่างชัดเจนว่า การใช้ LLM เพียงอย่างเดียว (LLM-only) แม้จะสามารถสร้างเรื่องราวที่น่าสนใจได้ แต่ก็ประสบปัญหาสำคัญในด้าน การปฏิบัติตามกฎ (Rule Adherence), การรักษาสถานะ (State Integrity) , และ ความยืดหยุ่นในการตอบสนอง (Flexibility / Railroading) ซึ่งเป็นหัวใจหลักของประสบการณ์ Tabletop RPG ผลลัพธ์ที่ได้มักเป็นเพียง "ผู้ช่วยเล่าเรื่อง" ไม่ใช่ "ผู้ดำเนินเกม" ที่แท้จริง
2. Hybrid คือทางออก: แนวทางที่ประสบความสำเร็จมากกว่า คือการใช้ สถาปัตยกรรมแบบผสมผสาน (Hybrid Architecture) ดังที่เห็นในงานของ Jørgensen et al. (2024) ซึ่งใช้ Multi-Agent และงานวิจัยกลุ่ม Tool-Assisted (2024) ที่ใช้ RAG และ Function Calling แนวทางเหล่านี้ชี้ให้เห็นว่า การ แยกส่วนการเล่าเรื่อง (Narrative - LLM) ออกจาก ส่วนกลไกเกม (Mechanics - Code/Tools) ช่วยให้ระบบมีความน่าเชื่อถือ (Reliability) และความสม่ำเสมอ (Consistency) สูงขึ้นอย่างมีนัยสำคัญ
3. การนำมาปรับใช้ในโครงการนี้ (Justification for "Two-Path"): จากบทเรียนข้างต้น โครงการนี้จึงเลือกใช้ สถาปัตยกรรมแบบ "Two-Path" (Hybrid-Arbitrated) ซึ่งเป็นการนำหลักการ Hybrid มาปรับใช้ให้เหมาะสมกับขอบเขตของ Lite 5e (ภาคผนวก ก) โดย

1) Path A (Fixed Actions): สำหรับการกระทำที่มีกฎชัดเจนใน Lite 5e (เช่น การโจมตี) ระบบจะใช้ Rules Engine (Symbolic Component) เป็นหลัก ในการคำนวณผลลัพธ์ เพื่อรับประกันความถูกต้อง 100% ตามที่งานวิจัยกลุ่ม Tool-Assisted แนะนำ

2) Path B (Creative Actions): สำหรับการกระทำที่ต้องอาศัยการตีความ หรืออยู่นอกเหนือกฎ Lite 5e (เช่น การสำรวจ, การไขปริศนา) ระบบจะใช้ LLM (ในฐานะ DM) (Neural Component) เป็นผู้ตัดสิน (Arbitrator) ในการกำหนด DC หรือเงื่อนไข ก่อนส่งให้ Rules Engine ทอยลูกเต๋า ซึ่งช่วยรักษาความยืดหยุ่นและความสร้างสรรค์ที่ LLM มี แต่ยังคงอยู่ภายใต้กรอบของกลไกเกม

สถาปัตยกรรม "Two-Path" นี้จึงเป็นการผสานจุดแข็งของ LLM (ความคิดสร้างสรรค์) และ Symbolic System (ความแม่นยำ) เพื่อเอาชนะข้อจำกัดที่พบในงานวิจัยก่อนหน้านี้ และสร้าง AI Dungeon Master ที่มีความสามารถรอบด้าน เหมาะสมกับผู้เล่นกลุ่มแคชวลตามเป้าหมายของโครงการ

ลำดับ	แผนดำเนินงาน	2568				2569				
		ก.ย.	ต.ค.	พ.ย.	ธ.ค.	ม.ค.	ก.พ.	มี.ค.	เม.ย	พ.ค.
6.	บูรณาการระบบเป็น Prototype-B (ผนวก ฟังก์ชันเสริม ASR/TTS) และ ทดสอบเบื้องต้น									
7.	ทดลองใช้งานระบบ (Pilot Test) และ รวบรวมผลตอบรับ									
8.	ปรับปรุงระบบขั้นสุดท้าย และจัดทำ รายงานฉบับสมบูรณ์									

ตารางที่ X แผนภูมิแกนต์

3.2 เครื่องมือและเทคโนโลยีที่ใช้

การพัฒนาโครงการนี้อาศัยเครื่องมือซอฟต์แวร์และเทคโนโลยีที่หลากหลาย ซึ่งถูกคัดเลือกมาโดยคำนึงถึงเป้าหมายหลักคือการสร้างแอปพลิเคชันที่ทำงานได้ด้วยตนเองบนหลายแพลตฟอร์ม (Cross-platform) และง่ายต่อการนำไปใช้งาน

1. ภาษาโปรแกรมและเฟรมเวิร์ก Dart เป็นภาษาหลักในการพัฒนา โดยใช้เฟรมเวิร์ก Flutter ในการสร้างแอปพลิเคชันส่วนหน้า (Frontend) เหตุผลที่เลือกใช้ Flutter เนื่องจากมีจุดแข็งในการใช้ฐานโค้ดเดียว (Single Codebase) แต่สามารถคอมไพล์เป็นแอปพลิเคชัน ที่ทำงานได้บนหลายระบบปฏิบัติการ เช่น Windows, macOS, Android และ iOS ซึ่งตอบโจทย์เป้าหมายของโครงการที่ต้องการให้ผู้ใช้สามารถเข้าถึงได้ง่าย
2. ส่วนประมวลผลหลัก (Backend Logic) โลจิกหลักของเกม เช่น Rules Engine และ State Tracker จะถูกพัฒนาด้วยภาษา Dart เช่นกัน เพื่อให้เป็นส่วนหนึ่งของแอปพลิเคชัน Flutter โดยตรง แนวทางนี้ช่วยลดความซับซ้อนของสถาปัตยกรรมลงอย่างมาก โดยไม่จำเป็นต้องมีการตั้งเซิร์ฟเวอร์แยกต่างหาก และทำให้แอปพลิเคชันสามารถทำงานได้อย่างสมบูรณ์ในตัวเอง (Self-contained)
3. การรู้จำเสียงพูด (ASR) (ฟังก์ชันเสริม) ใช้โมเดลโอเพนซอร์สสำเร็จรูปที่สามารถทำงานบนอุปกรณ์ได้โดยตรง (On-device) โดยจะมุ่งเน้นการบูรณาการไลบรารีเข้ากับ Flutter เพื่อเป็นทางเลือก (Alternative Input) ในการป้อนข้อมูล นอกเหนือจากการพิมพ์ และเพื่อรักษาความเป็นส่วนตัวของข้อมูลเสียง

4. แบบจำลองภาษาขนาดใหญ่ (LLM) เรียกใช้งานผ่าน REST API ไปยังผู้ให้บริการภายนอก โดยจะเลือกใช้โมเดลที่มีค่าใช้จ่ายไม่สูง เช่น Gemini Flash การเชื่อมต่อจะพัฒนาบน Flutter โดยใช้แพ็คเกจสำเร็จรูป เช่น flutter_gemini เพื่อจัดการการสื่อสารกับ API ซึ่งช่วยลดความซับซ้อนในการพัฒนาและทำให้สามารถมุ่งเน้นไปที่การสร้างตรรกะของเกมได้โดยตรง
5. การแปลงข้อความเป็นเสียง (Text-to-Speech - TTS) จะมีการเพิ่มฟังก์ชันเสริมในการอ่านออกเสียงข้อความบรรยายจาก AI DM ด้วยเสียงสังเคราะห์แบบพื้นฐาน โดยอาศัยไลบรารีมาตรฐานที่มีใน Flutter เพื่อเพิ่มทางเลือกในการรับรู้ข้อมูลของผู้เล่นโดยไม่เพิ่มความซับซ้อนให้กับระบบมากนัก

3.3 การออกแบบและพัฒนาระบบ

หัวใจของระเบียบวิธีวิจัยนี้คือการออกแบบสถาปัตยกรรมซอฟต์แวร์ที่สามารถบูรณาการเทคโนโลยีต่างๆ เข้าด้วยกันได้อย่างมีประสิทธิภาพ

3.3.1 ภาพรวมสถาปัตยกรรมระบบ (System Architecture Overview)

ระบบ AI Dungeon Master ถูกออกแบบตามสถาปัตยกรรมแบบ "Two-Path" (Hybrid-Arbitrated) ซึ่งเป็นการปรับปรุงแนวคิด Hybrid Architecture ให้เหมาะสมกับการเล่น Tabletop RPG ที่ต้องการทั้งความแม่นยำของกฎและความยืดหยุ่นในการเล่าเรื่อง สถาปัตยกรรมนี้มีหัวใจสำคัญคือ "Router" (ส่วนขยายของ Command Parsing) ที่ทำหน้าที่วิเคราะห์คำสั่งของผู้เล่นและเลือกเส้นทางการประมวลผลที่เหมาะสม โดยอ้างอิงจาก ชุดกติกา Lite 5e (ภาคผนวก ก) ดังนี้

Data Flow ในสถาปัตยกรรม "Two-Path"

1. รับอินพุตจากผู้เล่น (Player Input) กระบวนการเริ่มต้นเมื่อผู้เล่นป้อนคำสั่งเข้ามาในระบบ (ผ่านการพิมพ์ หรือฟังก์ชันเสริม ASR)
2. การประมวลผลอินพุต (Input Processing) ระบบแปลงอินพุต (ถ้าเป็นเสียง) ให้เป็นข้อความดิบ (Raw Text)
3. การวิเคราะห์และกำหนดเส้นทางคำสั่ง (Command Parsing & Routing - The "Router"): ขั้นตอนนี้คือหัวใจหลักของสถาปัตยกรรม "Two-Path" ระบบจะวิเคราะห์ข้อความดิบเพื่อ
 - 1) สกัดเจตนา (Intent) และองค์ประกอบ (Entities) เช่น { action: "attack", target: "goblin", weapon: "sword" }
 - 2) ตรวจสอบกับ Lite 5e Rulebook Router จะตรวจสอบว่า action ที่สกัดได้ตรงกับ "Fixed Action" (การกระทำที่มีกฎตายตัว) ที่นิยามไว้ในภาคผนวก ก (เช่น Attack, Move, Use Ability) หรือไม่
 - 3) กำหนดเส้นทาง (Routing)
 - (1) ถ้าเป็น Fixed Action: ระบบจะเลือก Path A (Rule-First)

- (2) ถ้าไม่ใช่ Fixed Action (เป็น Creative Action): ระบบจะเลือก Path B (LLM-First)

4. การประมวลผลตามเส้นทาง (Path Processing)

Path A: Rule-First (สำหรับ Fixed Actions)

1) ส่ง Intent ไปยัง Rules Engine ข้อมูลที่มีโครงสร้าง (Intent) จากขั้นตอนที่ 3 จะถูกส่งไปให้ Rules Engine โดยตรง

2) การบังคับใช้กฎ (Rule Enforcement) Rules Engine ทำการคำนวณผลลัพธ์ตามกลไก Lite 5e อย่างแม่นยำ เช่น ทำการสุ่มตัวเลข (RNG) เพื่อจำลองการทอย d20, เปรียบเทียบผลลัพธ์กับค่า AC, คำนวณความเสียหาย (Damage), และ อัปเดต Game State Tracker. ผลลัพธ์ที่ได้ (เช่น "Hit, 7 damage") จะถูกเก็บไว้

3) การดึงข้อมูลบริบท (Context Retrieval - RAG) ระบบ RAG ดึงข้อมูลสำคัญเพื่อสร้าง Prompt สำหรับ LLM

- (1) ผลลัพธ์จาก Rules Engine (จาก 2)
- (2) สถานะเกมล่าสุด (Dynamic Context) จาก State Tracker (เช่น Player HP: 12/15)
- (3) ประวัติการสนทนา (Historical Context)
- (4) กฎที่เกี่ยวข้อง (Static Context) (ถ้ามี, เช่น หากใช้เวทมนตร์ Rules Engine อาจระบุว่า ต้องดึงกฎ "Magic Missile" มาประกอบการบรรยาย)

4) การสร้าง Prompt และส่งให้ LLM (Narrator) ข้อมูลทั้งหมดจาก RAG ถูกประกอบเป็น Prompt และส่งให้ LLM

5) การสร้างคำบรรยาย (Narrative Generation): LLM ทำหน้าที่ บรรยายผลลัพธ์ ที่เกิดขึ้นตาม ข้อมูลบริบทที่ได้รับ (เช่น "ดาบของคุณฟันเข้าอย่างจัง! ก๊อบลินได้รับความเสียหาย 7 หน่วย")

Path B: LLM-First (สำหรับ Creative Actions)

1) การดึงข้อมูลบริบท (Context Retrieval - RAG) ระบบ RAG ดึงข้อมูลเพื่อสร้าง Prompt สำหรับ LLM (DM)

- (1) ข้อความดิบ (Raw Text) จากขั้นตอนที่ 2
- (2) สถานะเกมล่าสุด (Dynamic Context) จาก State Tracker
- (3) ประวัติการสนทนา (Historical Context)
- (4) กฎพื้นฐาน 3 ค่าสถานะ (PHYS/MENT/SOC) (Static Context) จาก Lite 5e เพื่อให้ LLM ใช้เป็นกรอบในการตัดสินใจ

2) การสร้าง Prompt และส่งให้ LLM (DM Arbitrator) ข้อมูลจาก RAG ถูกประกอบเป็น Prompt และส่งให้ LLM เพื่อ ทำการตัดสินใจ (Arbitrate) ในฐานะ DM

3) การตัดสินใจโดย LLM (DM Arbitration) LLM ตีความคำสั่ง Creative Action และกำหนดผลลัพธ์ที่เป็นไปได้ ซึ่งอาจจะเป็น

(1) สำเร็จ/ล้มเหลวทันที (เช่น ผู้เล่นขอลองคุยกับกำแพง LLM อาจตอบว่า "กำแพงไม่ตอบสนอง")

(2) ต้องการทอยลูกเต๋า LLM จะกำหนด ค่าสถานะที่ต้องใช้ (PHYS/MENT/SOC) และ ค่าความยาก (DC) (เช่น "ต้องทอย MENT ให้ได้ 15 เพื่อเผาหีบ")

4) (ถ้าต้องทอยเต๋า) ส่ง Check/DC ไปยัง Rules Engine ระบบส่งข้อมูล (เช่น check: "MENT", dc: 15) ไปให้ Rules Engine

5) การทอยลูกเต๋า (Dice Roll) Rules Engine ทำการทอย d20 + Modifier ที่เกี่ยวข้อง และส่งผลลัพธ์ (Success/Failure) กลับมา

6) การสร้างคำบรรยาย (Narrative Generation): LLM รับผลลัพธ์ (จาก 4b.3 หรือ 4b.5) และ บรรยายผล ของ Creative Action นั้นๆ (เช่น "คุณพยายามจุดไฟเผาหีบ แต่ไม่ยังขึ้นเกินไป...")

7) (ถ้าจำเป็น) การอัปเดตสถานะ: หากผลลัพธ์จาก LLM มีผลกระทบต่อสถานะเกม (เช่น ผู้เล่นบาดเจ็บจากการพยายามเผาหีบ) ระบบจะส่งข้อมูลไปอัปเดต Game State Tracker

5. การแสดงผลลัพธ์ (Output to UI): ระบบนำคำบรรยายที่สร้างโดย LLM (จาก 4a.5 หรือ 4b.6), ผลการทอยลูกเต๋า (ถ้ามี), และข้อมูลสถานะที่อัปเดตแล้ว มารวมกันเพื่อแสดงผลบน UI. นอกจากนี้ อาจมีการใช้ TTS (ฟังก์ชันเสริม) อ่านคำบรรยาย

หมายเหตุ: สถาปัตยกรรม "Two-Path" นี้ ต้องการ แผนภาพ (Diagram) ใหม่ เพื่อแสดงการไหลของข้อมูลและการตัดสินใจของ Router อย่างชัดเจน ซึ่งจะถูกสร้างขึ้นเพื่อประกอบรายงานฉบับสมบูรณ์

3.3.2 การพัฒนาส่วนประกอบหลัก (Development of Key Modules)

1. Rules Engine & Game State Tracker พัฒนาเป็นส่วนหนึ่งของแอปพลิเคชัน Flutter ด้วยภาษา Dart โดย Game State Tracker จะถูกออกแบบเป็นคลาสหลัก (e.g., GameState) ที่เก็บข้อมูลทั้งหมดในรูปแบบของ Class Object ที่มีโครงสร้างชัดเจน เช่น PlayerCharacter, Enemy, Item ซึ่งทำให้การจัดการข้อมูลเป็นระบบและง่ายต่อการแก้ไข ส่วน Rules Engine จะถูกสร้างเป็นกลุ่มของฟังก์ชันเชิงตรรกะ (e.g., calculateAttackOutcome()) ที่รับข้อมูลสถานะปัจจุบันเป็นอินพุตและส่งคืนผลลัพธ์โดยไม่มีการเปลี่ยนแปลงสถานะโดยตรง (Pure Functions) เพื่อให้ง่ายต่อการทดสอบและรับประกันความถูกต้อง

2. LLM Integration and Prompt Engineering ส่วนนี้จะเน้นที่การออกแบบ "Prompt Template" ที่มีประสิทธิภาพ โดยใช้หลักการที่ศึกษามาในการแบ่งโครงสร้าง Prompt ด้วย Tag ที่ชัดเจน (เช่น <context>, <rules>, <history>, <user_action>) เพื่อชี้แนะให้ LLM สามารถแยกแยะและเข้าใจบริบทต่างๆ ได้ดีที่สุด ซึ่งจะนำไปสู่การสร้างผลลัพธ์ที่สอดคล้องกับกฎและสถานะของเกม ลดโอกาสการเกิด Hallucination

3. User Interface (UI) Development พัฒนาระบบ Widget ของ Flutter โดยเน้นการออกแบบที่สะอาดและไม่ซับซ้อน (Minimalist Design) โดยให้ความสำคัญกับลำดับชั้นของข้อมูล (Information Hierarchy) เช่น การแสดงผลค่าพลังชีวิตและค่าสถานะหลักให้เห็นได้ชัดเจนตลอดเวลา ในขณะที่ส่วนของบันทึกเหตุการณ์ (Log) สามารถเลื่อนดูย้อนหลังได้

3.4 แหล่งข้อมูลและความรู้ของระบบ (System Knowledge and Data Sources)

1. ชุดกติกา Lite 5e คณะผู้จัดทำได้ออกแบบและสร้างเอกสารสรุปกฎ Lite 5e ขึ้นมาโดยเฉพาะ ซึ่งจะทำหน้าที่เป็นเอกสารข้อกำหนด (Specification Document) สำหรับการพัฒนาระบบทั้งหมดใน Rules Engine

2. เนื้อเรื่องและสถานการณ์ (Scenario Content): สำหรับต้นแบบ (Prototype) ระบบจะถูกออกแบบให้สามารถดำเนินเรื่องราวไปได้อย่างต่อเนื่องในลักษณะ Mission-based คือเมื่อจบภารกิจหนึ่ง ผู้เล่นจะสามารถรับภารกิจใหม่เพื่อเล่นต่อไปได้เรื่อยๆ โดยไม่มีจุดจบที่ตายตัว เพื่อรองรับการเล่นแบบปลายเปิด (Open-ended) และเพื่อใช้เป็นสถานการณ์ทดสอบหลักสำหรับ Pilot Test

3.5 การทดสอบและประเมินผล (System Testing and Evaluation)

การทดสอบระบบเป็นขั้นตอนสำคัญเพื่อรับประกันคุณภาพของต้นแบบ โดยจะแบ่งออกเป็น 2 ระยะหลัก เพื่อวัดผลเกณฑ์ชี้วัดความสำเร็จที่กำหนดไว้ใน Section 1.4.3

1. การทดสอบเชิงหน้าที่ (Functional Testing): เป็นการทดสอบเพื่อรับประกันความถูกต้องของโมดูลหลัก โดยเฉพาะ Rules Engine (เพื่อวัดผล Rule Adherence และ State Integrity). จะใช้วิธีการเขียน Unit Test ซึ่งเป็นการสร้างชุดคำสั่งอัตโนมัติเพื่อทดสอบการทำงานของฟังก์ชันต่างๆ ในสถานการณ์จำลอง (Scripted Scenarios) โดยอ้างอิงตามกฎ Lite 5e (ภาคผนวก ก). ตัวอย่าง Test Case เช่น

1) test_attack_roll(phys_bonus=2, target_ac=14) ต้องคืนค่า "Hit" ก็ต่อเมื่อผลทอย d20 $\geq$ 12.

2) test_critical_hit_damage(weapon_die='1d6', phys_bonus=2): ต้องคืนค่าความเสียหายเป็นสองเท่าของลูกเต๋า + bonus เมื่อผลทอย d20 = 20.

3) test_skill_check(stat_bonus=1, dc=13) ต้องคืนค่า "Success" ก็ต่อเมื่อผลทอย d20 $\geq$ 12.

4) test_hp_update(initial_hp=20, damage=7) ต้องอัปเดตค่า HP ใน GameState เป็น 13 อย่างถูกต้อง. แนวทางนี้มีความสำคัญอย่างยิ่งในการรับประกันว่าตรรกะหลักของเกมทำงานได้ถูกต้อง 100% ตามเกณฑ์ Rule Adherence และ State Integrity ก่อนนำไปบูรณาการกับ LLM

2. การทดลองใช้งานระบบ (Pilot Test) หลังจากบูรณาการทุกส่วนเป็น Prototype-B แล้ว จะมีการจัดทดลองใช้งานระบบกับกลุ่มเป้าหมาย 2-3 คน (คัดเลือกให้มีพื้นฐานหลากหลาย เช่น ผู้คุ้นเคยเกมกระดานแต่ไม่เคยเล่น D&D และผู้ไม่เคยเล่นเลย) เพื่อ ประเมินประสบการณ์ใช้งานจริงและวัดผลเกณฑ์ความสำเร็จจากมุมมองผู้ใช้

กระบวนการทดสอบจะแบ่งเป็น 3 ช่วง: 1) Onboarding, 2) ทดลองเล่น Scenario, และ 3) Feedback Collection

1) แบบสอบถามความพึงพอใจ (Likert Scale 1-5) ใช้ประเมินเกณฑ์ชี้วัดหลักจากมุมมองผู้ใช้ ตัวอย่างคำถามเช่น

Q1 (Rule Adherence Perception): "การตัดสินใจผลของการกระทำ (เช่น การโจมตี) รู้สึกว่า 'ยุติธรรม' และ 'สอดคล้อง' กับกฎ Lite 5e หรือไม่?"

Q2 (State Integrity Perception) "ข้อมูลตัวละคร (HP, Zone) บนหน้าจอ 'ถูกต้อง' และ 'อัปเดตทันที' หรือไม่?"

Q3 (Flow Management) "การเล่นดำเนินไป 'ราบรื่น' หรือไม่? มีช่วงที่ 'ติดขัด' หรือ 'ไม่รู้จะทำอะไรต่อ' หรือไม่?"

Q4 (Narrative - Path A) "คำบรรยายผลลัพธ์ของการกระทำตามกฎ (เช่น ผลโจมตี) 'ชัดเจน' หรือไม่?"

Q5 (Narrative - Path B) "เมื่อลองทำอะไรนอกเหนือกฎ (เช่น ตำรวจ) คำตัดสิน/คำบรรยายของ AI 'สมเหตุสมผล' หรือไม่?"

Q6 (Usability) "UI (หน้าจอต่างๆ) 'ใช้งานง่าย' หรือไม่?"

2) การสัมภาษณ์สั้นๆ (Qualitative Feedback) ส่วนนี้สำคัญมาก เพื่อเข้าใจ เหตุผล เบื้องหลังคะแนน และเปรียบเทียบมุมมองระหว่างผู้เล่นที่มีประสบการณ์ต่างกัน.

ทั้งนี้ การประเมิน Usability จะมุ่งเน้น UI หลัก ไม่รวมฟังก์ชันเสริม ASR/TTS. ผลตอบรับทั้งหมดจะถูกนำมาวิเคราะห์เพื่อสรุปผลความสำเร็จของโครงการเทียบกับเกณฑ์ใน Section 1.4.3 และเป็นแนวทางปรับปรุงต่อไป

เอกสารอ้างอิง

บทความในรายงานการประชุมทางวิชาการและวารสาร

1. Triyason, T., et al., 2023, "**Exploring the Potential of ChatGPT as a Dungeon Master in Dungeons & Dragons Tabletop Game**", Proceedings of the 2023 International Conference on Digital Image and Multimedia (ICODIM), 1-3 December, Chiang Mai, Thailand, pp. 1-6.
2. Jørgensen, N. H., Tharmabalan, T., Aslan, M., & Merritt, T., 2024, "**Evaluating Multi-Agent Architectures for an AI Game Master in Dungeons & Dragons**", Extended Abstracts of the 2024 CHI Conference on Human Factors in Computing Systems, 11-16 May, Honolulu, HI, USA.
3. Lanchantin, J., et al., 2024, "**You Have Thirteen Hours in Which to Solve the Labyrinth: Enhancing AI Game Masters with Function Calling**", arXiv preprint arXiv:2409.06949.
4. Vaswani, A., et al., 2017, "**Attention Is All You Need**", Advances in Neural Information Processing Systems 30 (NIPS 2017), Long Beach, CA, USA, pp. 5998-6008.
5. Radford, A., et al., 2022, "**Robust Speech Recognition via Large-Scale Weak Supervision**", Proceedings of the 39th International Conference on Machine Learning (ICML), 17-23 July, Baltimore, Maryland, USA, pp. 18076-18093.
6. Sakellaridis, P., 2024, **AI Dungeon Master for DnD Railroads Players**, [Online], Available: <https://www.wargamer.com/dnd/ai-railroads-players> [2025, October 19].
7. Maassen, H., 2019, **D&D 5e Lite**, [Online], Available: <https://haraldmaassen.com/blog/2019/06/02/dd-5e-lite/> [2025, October 19].
8. Babakcode, 2024, **flutter_gemini 3.0.0**, [Online], Available: https://pub.dev/packages/flutter_gemini [2025, October 19].

ภาคผนวก ก

ชุดกติกาย่อ Lite 5e (ฉบับสำหรับทีลีส)

ชุดกติกาย่อ Lite 5e (ฉบับสำหรับโครงการ)

บทนำ

ยินดีต้อนรับนักผจญภัย! คู่มือกติกานี้ถูกออกแบบมาสำหรับผู้เล่นใหม่ และเพื่อวัตถุประสงค์ของโครงการนี้โดยเฉพาะ ระบบของเราได้รับแรงบันดาลใจจาก Dungeons & Dragons ฉบับที่ 5 แต่ได้ถูกปรับให้เรียบง่ายขึ้น เพื่อให้:

- ง่ายต่อการเรียนรู้
- เล่นได้อย่างรวดเร็ว
- เหมาะสมกับการโต้ตอบกับ AI Dungeon Master (LLM)

ไม่ว่าคุณจะเป็นนักรบที่เหวี่ยงดาบ, นักบวชผู้ร้องขอพลังจากทวยเทพ, หรือจอมเวทผู้ควบคุมความเป็นจริง กฎเหล่านี้จะมอบโครงสร้างที่ยุติธรรมให้กับการเล่นของคุณ ในขณะที่ยังคงรักษาอิสระในการดำเนินเรื่องราวไว้

วิธีการเล่น (How to Play)

หัวใจหลักของเกมนี้เรียบง่าย:

1. LLM/DM บรรยายสถานการณ์ (เช่น “ก๊อบลินตัวหนึ่งกระโดดออกมาจากหลังถ้ำ!”)
2. ผู้เล่นบอกสิ่งที่คุณต้องการจะทำ (เช่น “ฉันจะใช้ดาบฟันไปที่ก๊อบลิน!”)
3. ระบบทอยลูกเต๋าเพื่อดูว่าคุณทำสำเร็จหรือไม่ (เช่น d20 + PHYS เทียบกับ AC ของศัตรู)
4. ผลลัพธ์จะถูกบรรยายออกมา (เช่น “ดาบของคุณฟันเข้าอย่างจัง! ก๊อบลินได้รับความเสียหาย 7 หน่วย”)

พื้นฐานเรื่องลูกเต๋า (Dice Basics)

เราจะใช้ระบบ d20 เป็นหลัก:

- การตรวจสอบทักษะ (Skill Checks): d20 + ค่าโบนัสสถานะ เทียบกับ ค่าความยาก (DC)
- การโจมตี (Attack Rolls): d20 + ค่าโบนัสสถานะ เทียบกับ ค่าความป้องกัน (AC) ของศัตรู
- การคำนวณความเสียหาย (Damage Rolls): ทอยลูกเต๋าน้อยกว่า (d6, d8, etc.)

การทอยพิเศษ:

- ได้แต้ม 20 (Natural 20): สำเร็จโดยอัตโนมัติ (Critical Success) และสร้างความเสียหายสองเท่า (สำหรับการโจมตี)
- ได้แต้ม 1 (Natural 1): ล้มเหลวโดยอัตโนมัติ (Critical Failure) และอาจมีผลเสียตามมา

ค่าสถานะและค่าโบนัส (Stats & Modifiers)

เราได้รวมค่าความสามารถทั้ง 6 ของ D&D แบบดั้งเดิมให้เหลือเพียง 3 ค่าสถานะหลัก:

ค่าสถานะ	ครอบคลุม	ตัวอย่างการกระทำ
กายภาพ (PHYS)	พลัง, ความคล่องแคล่ว, ความทนทาน	โจมตี, ปีนป่าย, หลบหลีก, ผลัก
สติปัญญา (MENT)	ความฉลาด, เชวชาญปัญญา, การรับรู้	ค้นหาความรู้, สังเกตกับดัก, แก้ปริศนา
สังคม (SOC)	เสน่ห์, การโน้มน้าวใจ	โน้มน้าวใจ, โทก, ช่มชู้

- แต่ละค่าสถานะจะมี ค่าโบนัส (Modifier) ตั้งแต่ -2 ถึง +5 ซึ่งจะนำไปบวกกับการทอยลูกเต๋า

พลังชีวิต (Hit Points - HP)

- ตัวละครทั้งหมดเริ่มต้นที่ 20 HP ที่เลเวล 1
- ความเสียหายจะลดค่า HP
- HP เหลือ 0 = หมดสติ (Unconscious)
- หากไม่ได้รับการรักษา อาจนำไปสู่ความตายได้

ระบบการต่อสู้ (Combat System)

การต่อสู้เป็นแบบ ผลัดกันเล่น (Turn-based) และใช้ระบบ โซน (Zone-based)

ลำดับการเล่น (Turn Order)

- ผู้เล่น → ศัตรู → ผู้เล่น → ศัตรู ... ดำเนินไปเรื่อยๆ จนกว่าฝ่ายใดฝ่ายหนึ่งจะพ่ายแพ้หรือล่าถอย

การกระทำในแต่ละรอบ (Actions per Turn)

- ในรอบของคุณ คุณสามารถทำ 1 การกระทำ (Action) + 1 การเคลื่อนที่ (Move):
 - การกระทำ (Action): โจมตี (ระยะประชิด/ระยะไกล), ร่ายเวท (ถ้ามี), ใช้ความสามารถพิเศษ, ใช้ไอเทม (ดื่มยา, โยนขวด), มีปฏิสัมพันธ์กับสิ่งแวดล้อม (ถีบประตู, ดึงคันโยก)
 - การเคลื่อนที่ (Move): เคลื่อนที่ไปยังโซนอื่น (ดูหัวข้อถัดไป)

การเคลื่อนที่ (Movement)

เราไม่ใช้ตารางการต่อสู้ แต่จะแบ่งพื้นที่ออกเป็น โซน:

- โซนเดียวกัน (Same Zone): สามารถโจมตีระยะประชิดและระยะไกลได้
- โซนติดกัน (Adjacent Zone): สามารถโจมตีระยะไกลได้ แต่ระยะประชิดไม่ได้

- โซนไกล (Far Zone): สามารถโจมตีระยะไกลได้ แต่จะมีค่าเสียเปรียบ (Disadvantage - อาจจะยังไม่ implement ในต้นแบบ)
- ผู้เล่นสามารถเคลื่อนที่ได้ 1 โซนต่อรอบ เป็นการเคลื่อนที่ (Move)

การโจมตี (Attacking)

- ทอยโจมตี (Attack Roll): $d20 + \text{PHYS}$ เทียบกับ ค่าความป้องกัน (AC) ของศัตรู
- ทอยความเสียหาย (Damage Roll): ลูกเต๋าของอาวุธ (เช่น $1d6, 1d8$) + ค่าโบนัส (ถ้ามี)
- การโจมตีติดคริติคอล (Natural 20): ความเสียหายสองเท่า

ตัวอย่างอาชีพ (Example Classes)

- นักรบ (Fighter):
 - HP: 20
 - Attack: $1d6 + \text{PHYS}$
 - Ability: Second Wind → รักษา HP $1d8$ (1 ครั้ง/การต่อสู้)
- พาลาดิน (Paladin):
 - HP: 20
 - Attack: $1d6 + \text{PHYS}$
 - Ability: Smite → เพิ่มความเสียหาย $1d8$ (2 ครั้ง/วัน)
 - Ability: Lay on Hands → รักษา HP $1d8$ (1 ครั้ง/วัน)
- นักบวช (Cleric):
 - HP: 18
 - Attack: $1d6 + \text{PHYS}$
 - Ability: Pray (ทอย MENT เทียบกับ DC 13) → เลือกผล: รักษาพันธมิตร ($1d8$), โจมตีศัตรู ($1d8$), หรือขับไล่ผีดิบ
- จอมเวท (Mage):
 - HP: 15
 - Attack: ไม้เท้า ($1d6$)
 - Spells: 2 ครั้ง/วัน (รูปแบบอิสระ, LLM ทำหน้าที่กำหนด DC และผลกระทบ)

การพักผ่อนและเวลา (Rest & Time)

- พักสั้น (Short Rest):
 - ระยะเวลา: 1 ชั่วโมง
 - ฟื้นฟู HP $1d8$ + ค่าโบนัส (ถ้ามี)
 - ฟื้นฟูความสามารถที่ใช้ได้จำกัด (เช่น Second Wind)

- พัักยาว (Long Rest):
 - ระยะเวลา: 8 ชั่วโมง
 - ฟื้นฟู HP ทั้งหมด
 - ฟื้นฟูเวทมนตร์และความสามารถทั้งหมด
 - ต้องทำในที่ที่ปลอดภัย

สถานะผิดปกติ (Conditions)

- หมดสติ (Unconscious): HP เหลือ 0, ไม่สามารถกระทำการใดๆ ได้
- มึนงง (Stunned): ข้ามรอบของตัวเองในเทิร์นถัดไป
- ถูกพันธนาการ (Restrained): การทอยเต่าที่ใช้ค่า PHYS จะเสียเปรียบ (Disadvantage)
- ตาย (Dead): ออกจากเกม เว้นแต่จะถูกชุบชีวิต

การเพิ่มระดับ (Leveling Up)

(หมายเหตุ: เป็นทางเลือกสำหรับต้นแบบ อาจยังไม่ implement)

- +5 HP ต่อเลเวล
- เลือกระหว่าง: +1 ค่าสถานะ หรือ เพิ่มการใช้ความสามารถ/เวทมนตร์ใหม่
- ระบบ XP อย่างง่าย: 250 XP ต่อเลเวล